FPGA應用設計
低光影像增強電路

學生: 林鼎軒、楊秉豪、陳柏廷

OUTLINE

- 低光影像電路設計
  - 軟體演算法原理
  - 硬體架構圖
- FPGA設計
  - FPGA Block Design
  - PS PL Design

軟體演算法原理：暗通道先驗 (Dark Channel Prior)

核心步驟

1. 負片轉換： 將原始像素值反轉，便於後續處理。

2. 暗通道計算： 使用 3x3 卷積核尋找 RGB 三通道的區域最小值。

3. 透射率估算： 設定大氣光強度 A 與權重 omega (0.75) 計算透射率。

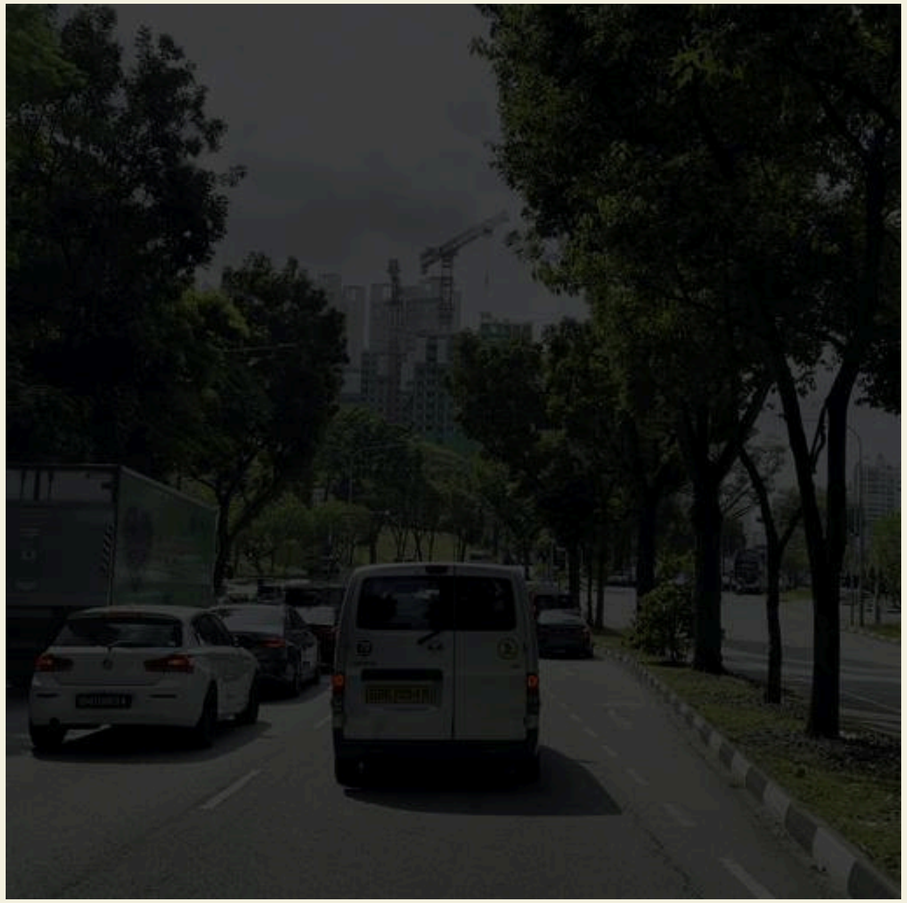
4. 影像恢復： 基於透射率公式還原影像細節，最後轉回正片。

$$R^c(x) = 255 - I^c(x)$$

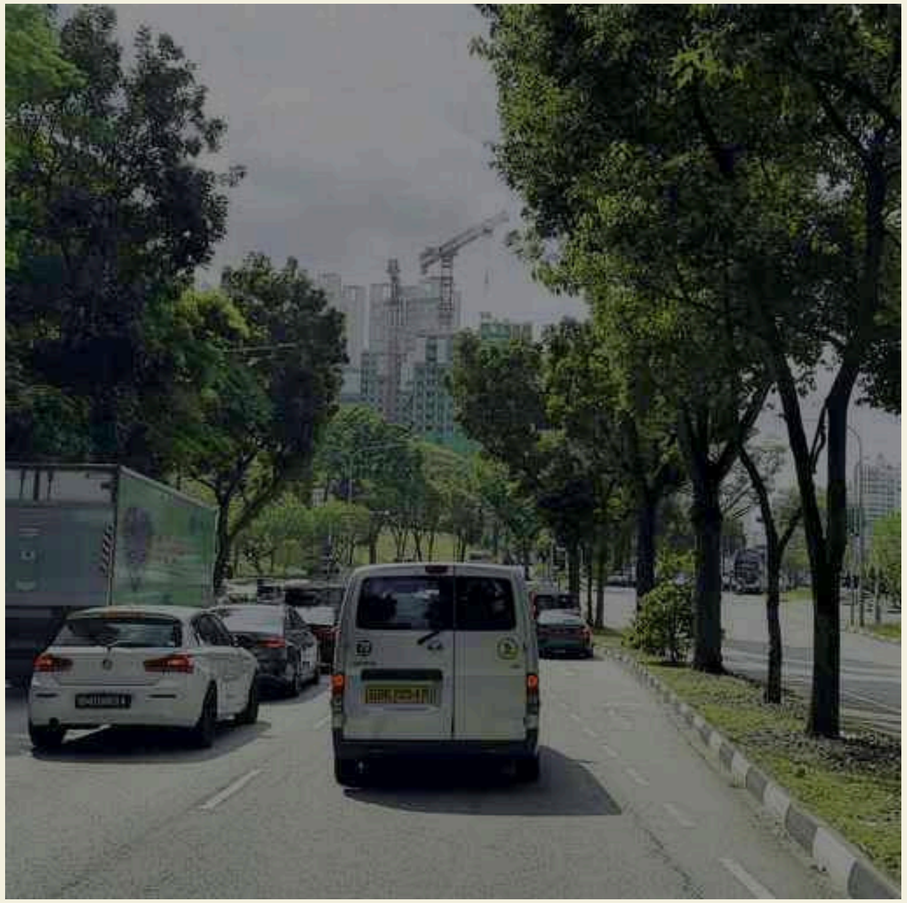
$$J^{dark}(x) = \min_{y \in \Omega(x)} \left(\min_{c \in \{r,g,b\}} R^c(y) \right)$$

$$t(x) = 1 - \frac{\omega \cdot J^{dark}(x)}{A}$$

$$J(x) = A - \frac{A - R(x)}{t(x)}$$



ori_image



Enhance_image



ori_image



Enhance_image



ori_image



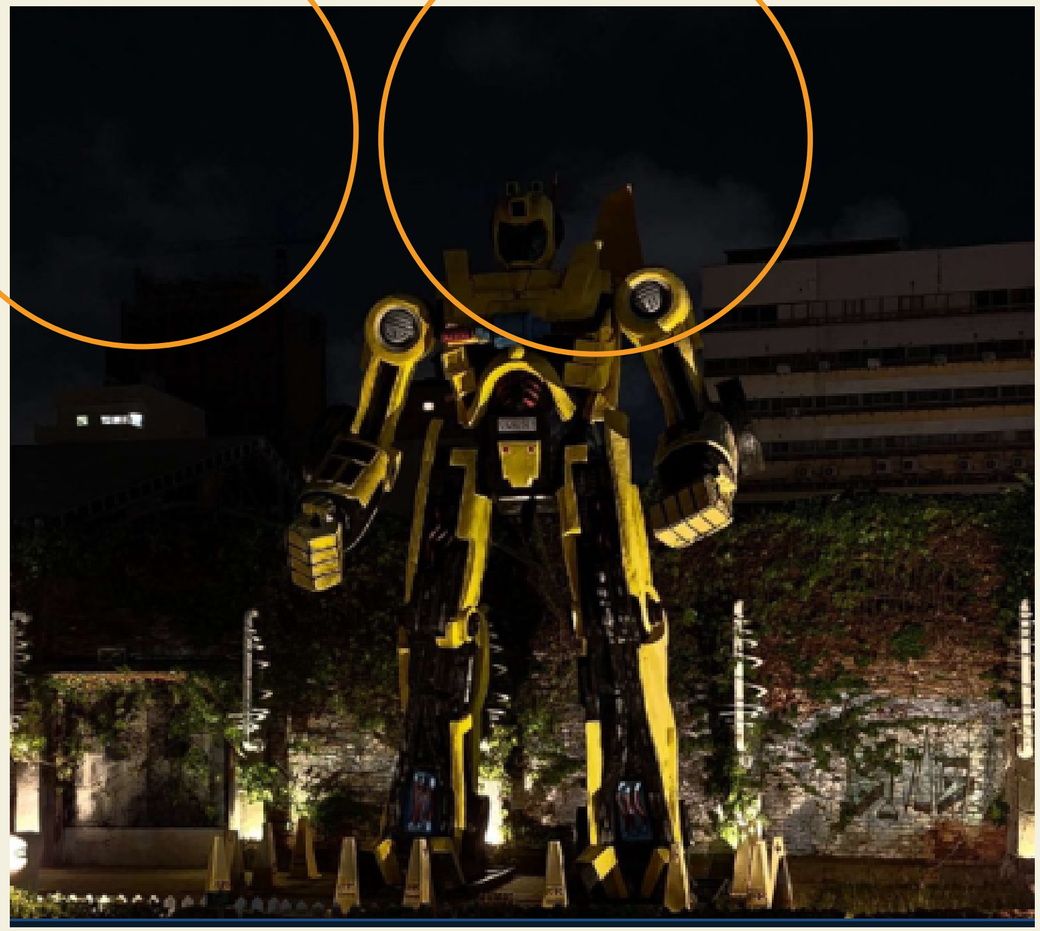
Enhance_image



ori_image



Enhance_image



ori_image



Enhance_image

系統設計流程

Python 前處理

負責影像裁切、縮放至固定大小 ，並轉換為 Hex 格式供硬體讀取。

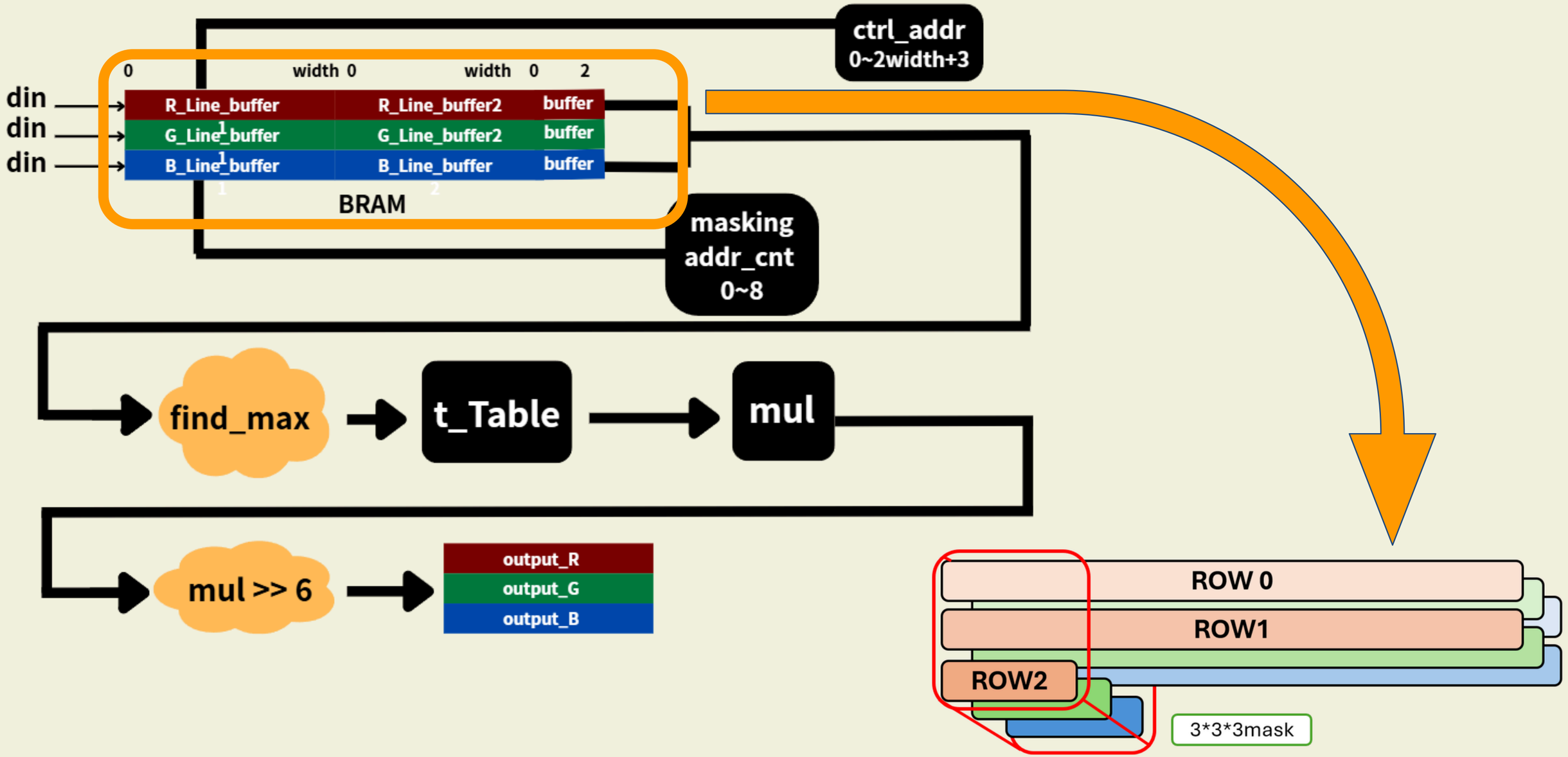
Verilog 硬體加速

核心演算法實現。包含記憶體管理、Pipeline 運算架構與數據處理模組。

後處理驗證

將硬體輸出的 Hex 數據轉換回 JPG 影像，進行視覺與數據驗證。

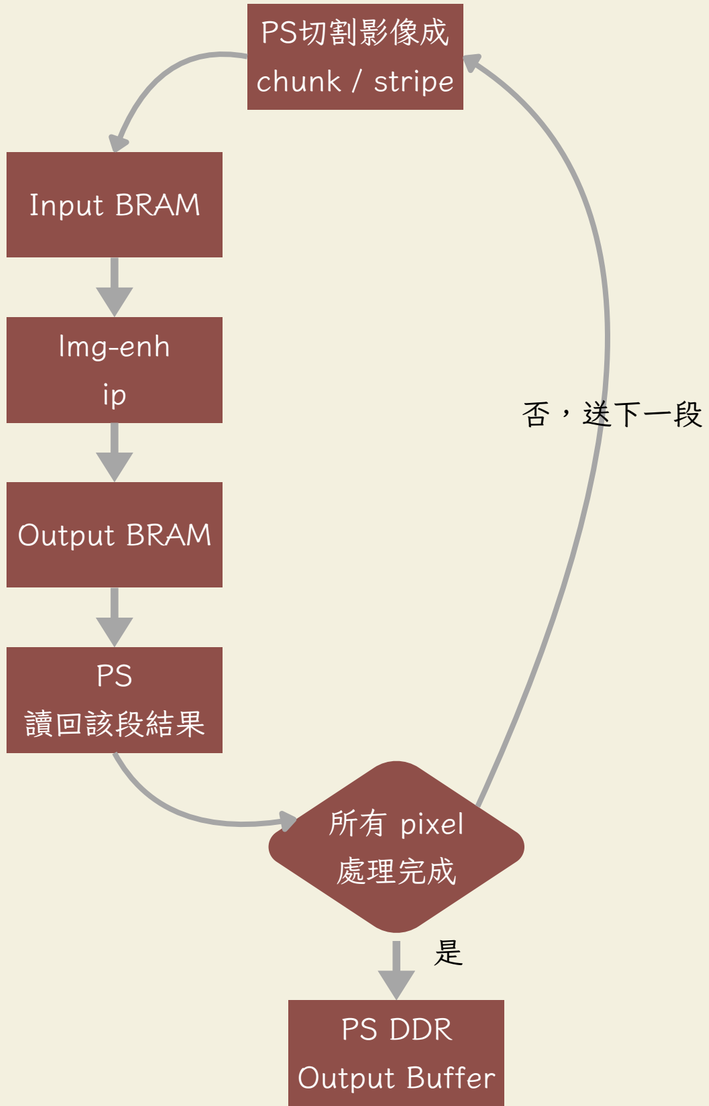
硬體流程圖


ctrl_addr
0~2width+3
din
din
din
0
width 0
width 0
2
R_Line_buffer
R_Line_buffer2
buffer
G_Line_buffer
G_Line_buffer2
buffer
B_Line_buffer
B_Line_buffer
buffer
BRAM
masking
addr_cnt
0~8
find_max
t_Table
mul
mul >> 6
output_R
output_G
output_B
ROW 0
ROW1
ROW2
3*3*3mask

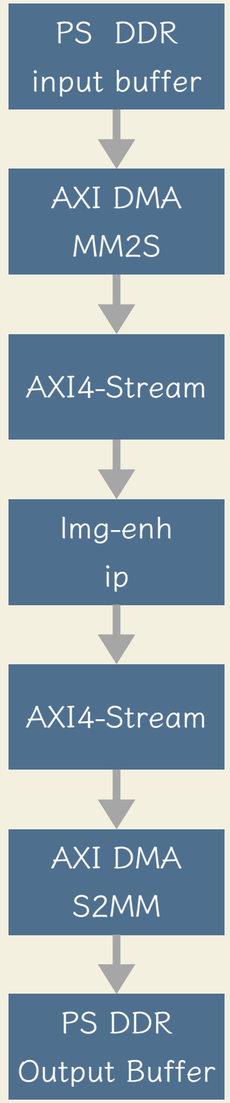
FPGA Block Design

BRAM vs DDR

BRAM 容量 ：630KB



DDR 容量： 512 MB



FPGA Block Design

AXI wrapper

```
module top_pipeline_axis_wrapper #(
    parameter integer OUTPUT_FIFO_ADDR_WIDTH = 6
)(
    input  wire        aclk,
    input  wire        aresetn,

    input  wire [31:0] s_axis_tdata,
    input  wire [3:0]  s_axis_tkeep,
    input  wire        s_axis_tvalid,
    output wire        s_axis_tready,
    input  wire        s_axis_tlast,

    output wire [31:0] m_axis_tdata,
    output wire [3:0]  m_axis_tkeep,
    output wire        m_axis_tvalid,
    input  wire        m_axis_tready,
    output wire        m_axis_tlast,

    output wire        busy,
    output wire        done,
    output reg         error_unexpected_tlast,
    output reg         error_missing_tlast,
    output reg         error_bad_tkeep,
    output reg         error_output_overflow
);

    localparam [1:0] ST_IDLE  = 2'd0;
    localparam [1:0] ST_RECV  = 2'd1;
    localparam [1:0] ST_FLUSH = 2'd2;
    localparam [1:0] ST_DRAIN = 2'd3;

    localparam [19:0] FRAME_PIXELS = 20'd262144;
    localparam [19:0] FRAME_LAST   = 20'd262143;
    localparam integer OUTPUT_FIFO_DEPTH = (1 << OUTPUT_FIFO_ADDR_WIDTH);
```

```
always @(posedge aclk) begin
    if (!aresetn) begin
        state                 <= ST_IDLE;
        accepted_count        <= 20'd0;
        produced_count        <= 20'd0;
        feed_valid            <= 1'b0;
        feed_data             <= 32'd0;
        feed_index            <= 20'd0;
        flush_index           <= FRAME_PIXELS;
        core_started          <= 1'b0;
        out_fifo_wr_ptr       <= {OUTPUT_FIFO_ADDR_WIDTH{1'b0}};
        out_fifo_rd_ptr       <= {OUTPUT_FIFO_ADDR_WIDTH{1'b0}};
        out_fifo_count        <= {(OUTPUT_FIFO_ADDR_WIDTH+1){1'b0}};
        done_pulse            <= 1'b0;
        error_unexpected_tlast <= 1'b0;
        error_missing_tlast   <= 1'b0;
        error_bad_tkeep       <= 1'b0;
        error_output_overflow <= 1'b0;
    end else begin
        done_pulse  <= 1'b0;
        if (out_fifo_push) begin
            out_fifo[out_fifo_wr_ptr] <= {(produced_count == FRAME_LAST), out_fifo_push_data};
            out_fifo_wr_ptr <= out_fifo_wr_ptr + 1'b1;
            produced_count <= produced_count + 1'b1;
        end else if (core_output_valid && fifo_full) begin
            error_output_overflow <= 1'b1;
        end

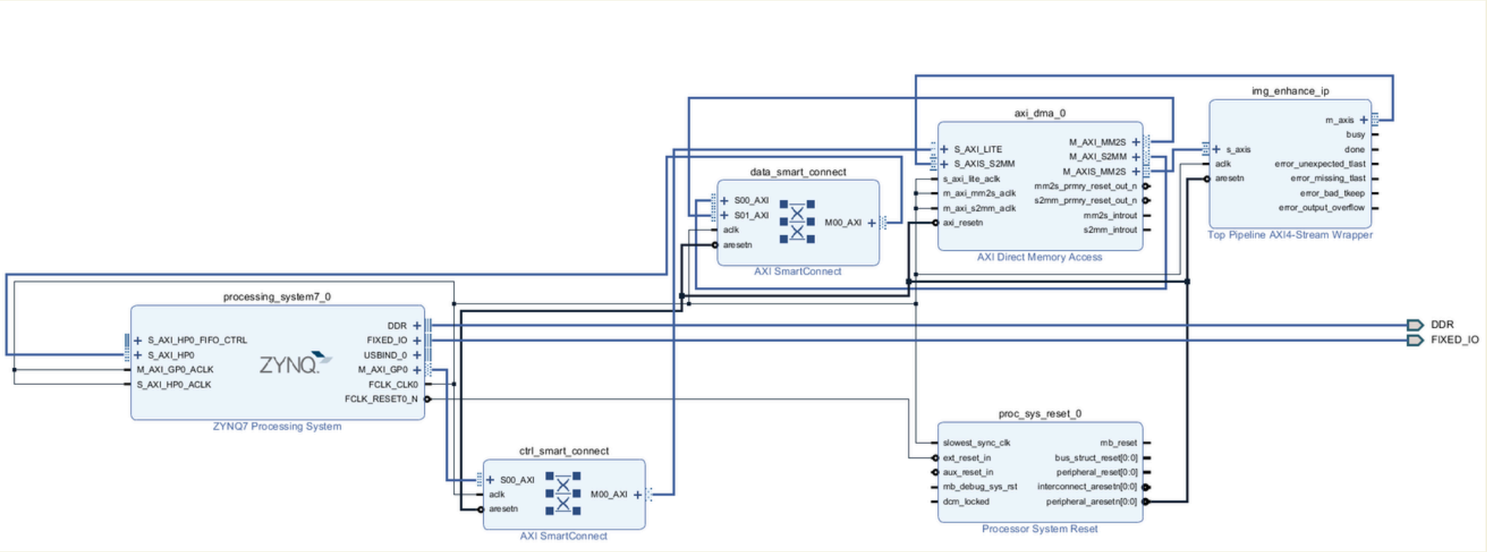
        if (out_fifo_pop) begin
            out_fifo_rd_ptr <= out_fifo_rd_ptr + 1'b1;
        end

        case ({out_fifo_push, out_fifo_pop})
            2'b10: out_fifo_count <= out_fifo_count + 1'b1;
            2'b01: out_fifo_count <= out_fifo_count - 1'b1;
            default: out_fifo_count <= out_fifo_count;
        endcase

        case (state)
            ST_IDLE: begin
                accepted_count <= 20'd0;
                produced_count <= 20'd0;
                feed_valid     <= 1'b0;
                feed_data      <= 32'd0;
                feed_index     <= 20'd0;
                flush_index    <= FRAME_PIXELS;
                core_started   <= 1'b0;
```

FPGA Block Design

Block Design& Utilization Overview



Block Design

Resource	Usage
LUT	11%
LUTRAM	5%
FF	7%
BRAM	5%
DSP	1%
BUFG	3%

Utilization

FPGA Block Design

PS端處理流程

PS 端前處理

讀取影像、轉成固定大小、轉成 Hex / 0x00RRGGBB、放入 DDR buffer

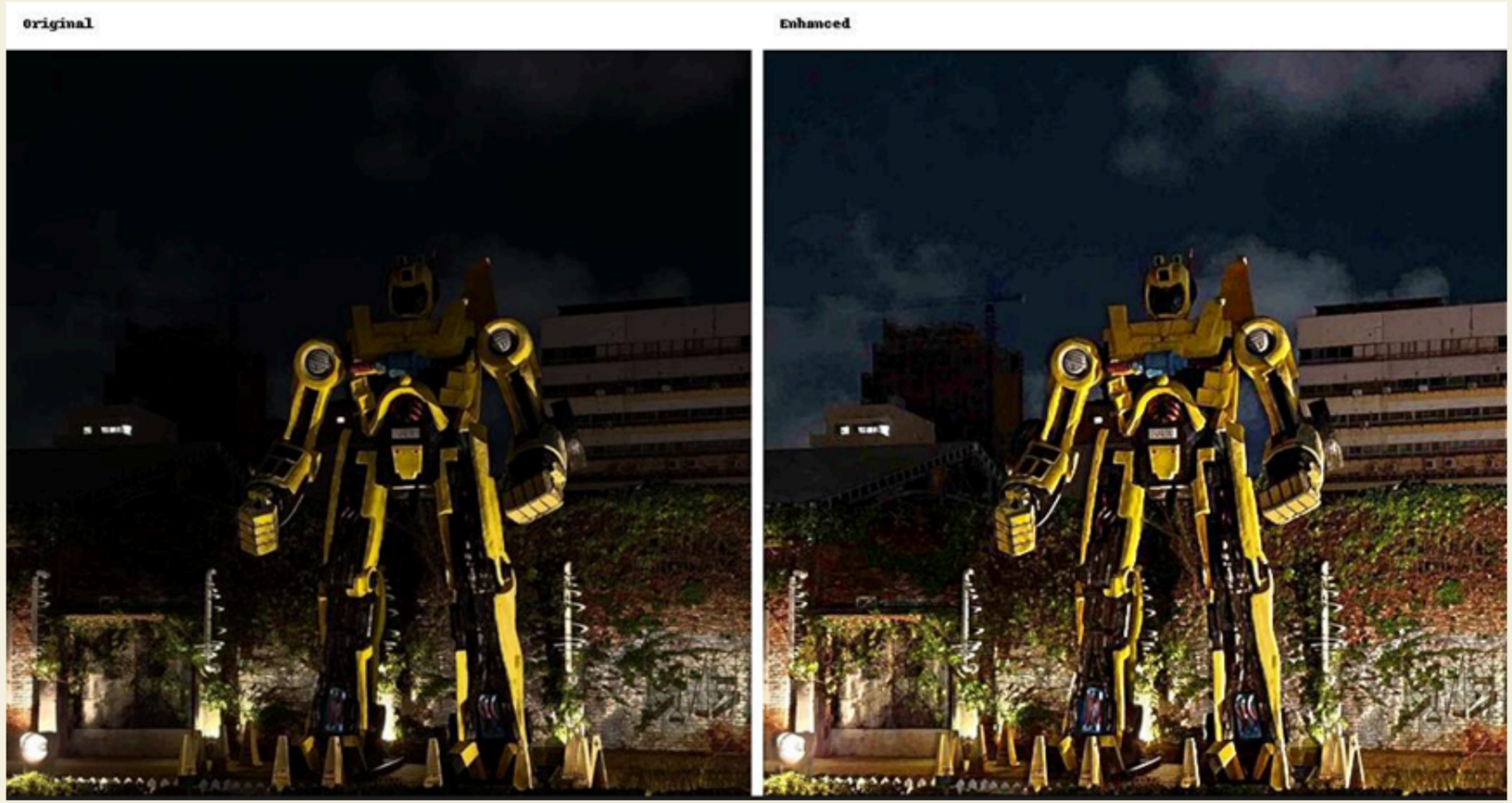
PL 端硬體加速

AXI DMA 傳輸、AXI4-Stream Wrapper、Image Enhance Core、Pipeline 運算

PS 端後處理與驗證

從 DDR output buffer 讀回、轉回 Hex / JPG、輸出比較圖、檢查結果

FPGA result



FPGA DMA+compute time: 7.589 ms, 131.77 FPS, 34.54 Mpixel/s, 131.77 MiB/s

使用公開資料集對演算法評測

資料集名稱	NIQE	Entropy	BRISQUE
DICM	3.68	6.86	32.25
LIME	3.76	6.85	37.06
MEF	3.89	6.91	37.25
NPE	3.38	7.46	26.88
VV	3.5	7.19	32.79
	<5為影像品質良好 Lower is Better	~=7為影像資訊量良好 Higher is Better	20 ~ 40為品質良好 Lower is Better

參考資料

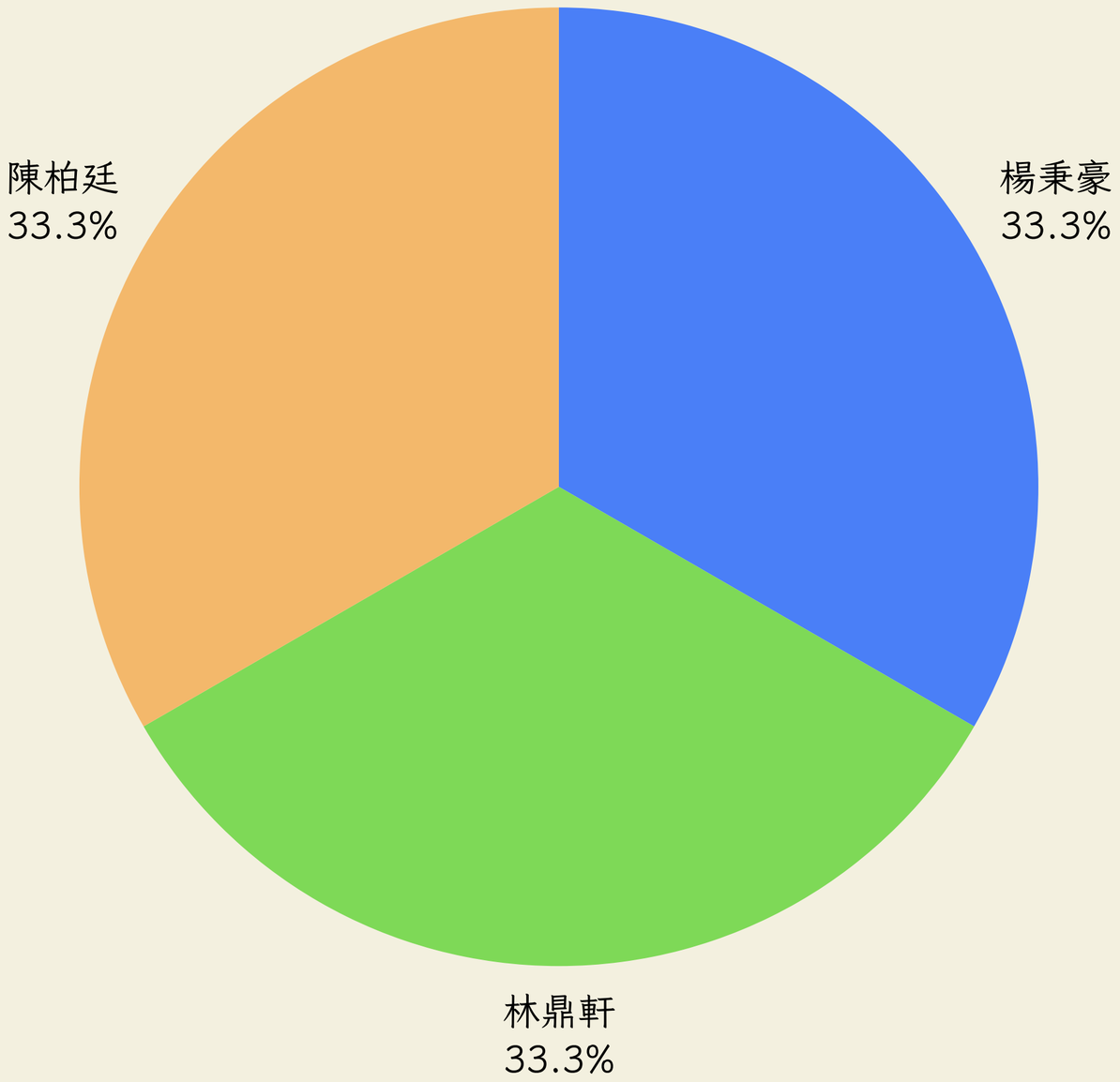
改良式暗通道先驗之單圖像去霧實現於FPGA
Single Image Dehazing Using Enhanced Dark Channel Prior Implemented on FPGA --- 曾子倫 國立台灣科技大學

single image haze removal using dark channel prior ---- Kaiming He; Jian Sun; Xiaoou Tang, 2009

Non-Local Image Dehazing --- Dana Berman, Tali Treibitz, Shai Avidan 2016

Fast Haze Removal for Nighttime Image Using Maximum Reflectance Prior (CVPR 2017)---Jing Zhang, Yang Cao, et al

貢獻度



Thank you

感謝聆聽 指導與提問